

Sprint Documentation

Sprint 15	Start Date: 14/04/2026	Final Date: 20/04/2026
Project Name:	civitas-backend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
João Pedro Chapichi		
João Antonio Gomes Cestonaro		

Sprint Backlog

Task #	Description	Start Date	Final date
#80	Validações de Tipo de Despesa	14/04/2026	20/04/2026

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
#80	Implementar validações no backend para os campos da entidade de tipo de despesa, assegurando: - Dados obrigatórios - Padronização de informações - Integridade relacional	João Pedro Chapichi, João Antonio Gomes Cestonaro	Completo	7 horas	7 horas

Sprint Description

Tarefa: Validações de Tipo de Despesa

Objetivo

Garantir que o cadastro e manutenção dos tipos de despesa possuam validações completas de integridade, padronização e regras de negócio, assegurando consistência no lançamento de despesas no sistema. Além disso, aplicar correções necessárias no sistema.

Descrição

Implementar validações no backend para os campos da entidade de tipo de despesa, assegurando:

Dados obrigatórios

Padronização de informações

Integridade relacional

Critérios de Aceite

◆ Campos Obrigatórios

Validar que os campos obrigatórios não estão vazios:

descricao

idUnidadeMedida (Analisar se campo deve permanecer)

solicitaUc (Analisar se campo deve permanecer)

situacao

idTipoCodigo

Validação de Descrição

Campo obrigatório

Mínimo de 3 caracteres

Máximo de 150 caracteres

Remover espaços extras (trim)

Padronizar (ex: evitar duplicidade por diferença de maiúsculo/minúsculo)

Não permitir descrições duplicadas no sistema

◆ Validação de Unidade de Medida

idUnidadeMedida deve existir no banco

Validar integridade da chave estrangeira

Garantir que a unidade seja válida para uso

◆ Validação de Tipo de Código

idTipoCodigo deve existir no banco

Validar integridade da chave estrangeira

◆ Validação de Solicitação de UC

Deve corresponder a um valor válido do enum (SolicitaUc)

Valores permitidos:

1 = Sim

2 = Não

Rejeitar valores fora do padrão

◆ Regra de Negócio (UC)

Quando SolicitaUc = Sim:

O sistema deve exigir preenchimento da UC no cadastro de despesa

Quando SolicitaUc = Não:

O campo UC deve ser opcional ou desabilitado

◆ Validação de Situação

Deve corresponder a um valor válido do enum (Situacao)

Valores permitidos:

1 = Ativo

2 = Inativo

Rejeitar valores fora do padrão

◆ Regra de Negócio (Situação)

Tipos de despesa INATIVOS:

Não devem aparecer para seleção no cadastro de despesas

Não podem ser utilizados em novos lançamentos

◆ Validação em Atualização

☐ Ao alterar situacao para INATIVO:

Garantir que não impacte lançamentos futuros

☐ Manter consistência dos dados após alteração

◆ Validação de Inativação

☐ Não permitir inativação de tipo de despesa que possua:

Despesas vinculadas

◆ Normalização e Boas Práticas

☐ Padronizar descrição (ex: primeira letra maiúscula)

☐ Validar dados antes de persistir no banco

☐ Implementar validações na camada SERVICE

[] Não depender apenas de DataAnnotations

[] Retornar mensagens claras ao usuário

Observações e Sugestões

Garantir que o tipo de despesa controle corretamente o comportamento das despesas

Evitar duplicidade de categorias (ex: "Energia" vs "energia")

Manter consistência entre regras de negócio e interface do sistema

Utilizar validações centralizadas para facilitar manutenção futura

Os erros retornados ao usuário devem vir em uma lista com todos os erros de uma só vez ao invés de mostrar um erro por vez.

Verifiquem o projeto antes do envio

Sprint Results

Passo a passo do que foi feito.

1. Adicionado `ExistsByDescricaoNormalized(string descricaoNormalizada, int? ignoreId = null)` para validar duplicidade de descrição e `HasDespesasVinculadas(int idTipoDespesa)` para bloquear inativação com despesas relacionadas. (prints/imagem01)
2. Implementado os novos métodos para verificar se possui descrição duplicada e normalizá-la antes de salvar no banco e verificar se possui despesas vinculadas ao `TipoDespesa`. (prints/imagem02)
3. Adicionado na `ITipoDespesaService` o `HasDespesasVinculadas(int idTipoDespesa)` para suportar a regra de inativação no controller/service. (prints/imagem03)
4. Criado novo arquivo `TipoDespesaValidationException` criando uma validação específica para a `TipoDespesa` e estrutura para retornar todos os erros de uma só vez. (prints/imagem04)
5. Implementado um override de `Create(TipoDespesaDTO)` e `Update(TipoDespesaDTO, int)` com fluxo: validar DTO => normalizar => validar regras comuns => validar regras de negócio. Normalização de Descrição (trim, remoção de espaços extras e padronização). Validações de obrigatórios, tamanho, enum (`SolicitaUC`, `Situacao`), FK (`IdUnidadeMedida`, `IdTipoCodigo`), unidade ativa e duplicidade. Regra de negócio para bloquear inativação quando houver despesas vinculadas. Lançamento de `TipoDespesaValidationException` com lista completa de erros. (prints/imagem05 até imagem11)
6. Tratamento de `TipoDespesaValidationException` em `Post(TipoDespesaDTO)` e `Put(int, TipoDespesaDTO)`, retorna 400 com `ResponseEnum.INVALID` e lista de erros em Data. Ajuste da regra no PATCH `situacao/{id}`, passou a bloquear inativação por despesas vinculadas (não por unidade de medida). (prints/imagem12 até imagem 14)
7. Em `ValidarRelacionamentosAsync(DespesaDTO)`, agora bloqueia lançamento novo quando `TipoDespesa` está INATIVO. (prints/imagem15)
8. Adicionado novo arquivo `TipoDespesaValidationEndpointsTests.cs`, testes de integração adicionados para validar normalização da descrição, retorno de múltiplos erros em uma única resposta, bloqueio de duplicidade case-insensitive, bloqueio de inativação com despesas vinculadas e bloqueio de lançamento com `TipoDespesa` inativo. (Não precisa adicionar imagem sobre isso, vai dar muitas)

```
Task<bool> ExistsByDescricaoNormalized(string descricaoNormalizada, int? ignoreId = null);
3 referências
Task<bool> HasDespesasVinculadas(int idTipoDespesa);
```

```
public async Task<bool> ExistsByDescricaoNormalized(string descricaoNormalizada, int? ignoreId = null)
{
    var query = _context.TiposDespesa
        .AsNoTracking()
        .Where(t => t.Descricao.Trim().ToUpper() == descricaoNormalizada);

    if (ignoreId.HasValue)
    {
        query = query.Where(t => t.Id != ignoreId.Value);
    }

    return await query.AnyAsync();
}

3 referências
public async Task<bool> HasDespesasVinculadas(int idTipoDespesa)
{
    return await _context.Despesas
        .AsNoTracking()
        .AnyAsync(d => d.IdTipoDespesa == idTipoDespesa);
}
```

```
Task<bool> HasDespesasVinculadas(int idTipoDespesa);
```

```
namespace Civitas.WebAPI.Services.Validation
{
    6 referências
    public class TipoDespesaValidationException : Exception
    {
        3 referências
        public TipoDespesaValidationException(IEnumerable<string> errors)
            : base("Os dados informados para o tipo de despesa são inválidos.")
        {
            Errors = errors.ToArray();
        }

        3 referências
        public IReadOnlyCollection<string> Errors { get; }
    }
}
```

```

using AutoMapper;
using Civitas.WebAPI.Data.Interfaces;
using Civitas.WebAPI.Objects.Dtos.Entities;
using Civitas.WebAPI.Objects.Enums;
using Civitas.WebAPI.Objects.Models;
using Civitas.WebAPI.Services.Interfaces;
using Civitas.WebAPI.Services.Validation;
using System.Globalization;
using System.Text.RegularExpressions;

namespace Civitas.WebAPI.Services.Entities
{
    /// <summary>
    /// Serviço responsável pela gestão das Categorias de Despesa (Configurações de Lançamento).
    /// </summary>
    /// <remarks>
    /// Finalidade:
    /// - Gerenciar os tipos de contas permitidos no sistema (Água, Luz, Telefone).
    /// - Centralizar a configuração de regras de preenchimento (se exige UC, qual unidade de medida usa).
    ///
    /// Dependências:
    /// - <see cref="ITipoDespesaRepository"/>: Acesso a dados.
    /// - <see cref="IMapper"/>: Mapeamento de objetos.
    /// </remarks>
    2 referências
    public class TipoDespesaService : GenericService<TipoDespesa, TipoDespesaDTO>, ITipoDespesaService
    {
        private static readonly CultureInfo PtBrCulture = new("pt-BR");

        private readonly ITipoDespesaRepository _tipoDespesaRepository;
        private readonly IUnidadeMedidaRepository _unidadeMedidaRepository;
        private readonly ITipoCodigoRepository _tipoCodigoRepository;
        private readonly IMapper _mapper;

        /// <summary>
        /// Inicializa o serviço de Tipos de Despesa.
        /// </summary>
        /// <param name="tipoDespesa">Repositório concreto de tipos de despesa.</param>
        /// <param name="mapper">Mapeador de objetos.</param>

```

```

    public TipoDespesaService(
        ITipoDespesaRepository tipoDespesa,
        IUnidadeMedidaRepository unidadeMedidaRepository,
        ITipoCodigoRepository tipoCodigoRepository,
        IMapper mapper)
        : base(tipoDespesa, mapper)
    {
        _tipoDespesaRepository = tipoDespesa;
        _unidadeMedidaRepository = unidadeMedidaRepository;
        _tipoCodigoRepository = tipoCodigoRepository;
        _mapper = mapper;
    }

    15 referências
    public override async Task Create(TipoDespesaDTO entityDTO)
    {
        ValidateDtoInstance(entityDTO);
        Normalize(entityDTO);

        var errors = ValidateCommonRules(entityDTO);
        await ValidateBusinessRules(entityDTO, errors);

        if (errors.Count > 0)
        {
            throw new TipoDespesaValidationException(errors);
        }

        var entity = _mapper.Map<TipoDespesa>(entityDTO);
        await _tipoDespesaRepository.Add(entity);

        entityDTO.Id = entity.Id;
    }

```

```

    24 referências
    public override async Task Update(TipoDespesaDTO entityDTO, int id)
    {
        ValidateDtoInstance(entityDTO);

        var existingTipoDespesa = await _tipoDespesaRepository.GetById(id);
        if (existingTipoDespesa is null)
        {
            throw new KeyNotFoundException($"Tipo de despesa com id {id} não encontrado.");
        }

        Normalize(entityDTO);

        var errors = ValidateCommonRules(entityDTO);
        await ValidateBusinessRules(entityDTO, errors, id, existingTipoDespesa);

        if (errors.Count > 0)
        {
            throw new TipoDespesaValidationException(errors);
        }

        var entity = _mapper.Map<TipoDespesa>(entityDTO);
        entity.Id = id;

        await _tipoDespesaRepository.Update(entity);

        entityDTO.Id = id;
    }

```

```

1 referência
public async Task<bool> ExisteUnidadesDeMedidaAtivas(int idTipoDespesa)
{
    return await _tipoDespesaRepository.ExisteUnidadesDeMedidaAtivas(idTipoDespesa);
}

2 referências
public async Task<bool> HasDespesasVinculadas(int idTipoDespesa)
{
    return await _tipoDespesaRepository.HasDespesasVinculadas(idTipoDespesa);
}

3 referências
private static void ValidateDtoInstance(TipoDespesaDTO? tipoDespesaDTO)
{
    if (tipoDespesaDTO is null)
    {
        throw new TipoDespesaValidationException(["O corpo da requisição é obrigatório."]);
    }
}

private static void Normalize(TipoDespesaDTO tipoDespesaDTO)
{
    var descricao = tipoDespesaDTO.Descricao?.Trim() ?? string.Empty;
    descricao = Regex.Replace(descricao, "\\s+", " ");

    if (!string.IsNullOrEmpty(descricao))
    {
        descricao = PtBrCulture.TextInfo.ToTitleCase(descricao.ToLower(PtBrCulture));
    }

    tipoDespesaDTO.Descricao = descricao;
}

```

```

2 referências
private static List<string> ValidateCommonRules(TipoDespesaDTO tipoDespesaDTO)
{
    var errors = new List<string>();

    if (string.IsNullOrWhiteSpace(tipoDespesaDTO.Descricao))
    {
        errors.Add("O campo Descrição é obrigatório.");
    }
    else
    {
        if (tipoDespesaDTO.Descricao.Length < 3)
        {
            errors.Add("O campo Descrição deve ter no mínimo 3 caracteres.");
        }

        if (tipoDespesaDTO.Descricao.Length > 150)
        {
            errors.Add("O campo Descrição deve ter no máximo 150 caracteres.");
        }
    }

    if (tipoDespesaDTO.IdUnidadeMedida <= 0)
    {
        errors.Add("O campo IdUnidadeMedida é obrigatório.");
    }

    if (tipoDespesaDTO.IdTipoCodigo <= 0)
    {
        errors.Add("O campo IdTipoCodigo é obrigatório.");
    }

    if (tipoDespesaDTO.SolicitaUc is not (SolicitaUc.Sim or SolicitaUc.Não))
    {
        errors.Add("SolicitaUc inválido. Valores permitidos: 1 (Sim) ou 2 (Não).");
    }

    if (tipoDespesaDTO.Situacao is not (Situacao.ATIVO or Situacao.INATIVO))
    {
        errors.Add("Situação inválida. Valores permitidos: 1 (Ativo) ou 2 (Inativo).");
    }
}

```



```

        return errors;
    }

    2 referências
    private async Task ValidateBusinessRules(
        TipoDespesaDTO tipoDespesaDTO,
        ICollection<string> errors,
        int? id = null,
        TipoDespesa? existingTipoDespesa = null)
    {
        if (tipoDespesaDTO.IdUnidadeMedida > 0)
        {
            var unidadeMedida = await _unidadeMedidaRepository.GetById(tipoDespesaDTO.IdUnidadeMedida);
            if (unidadeMedida is null)
            {
                errors.Add("A unidade de medida informada não existe.");
            }
            else if (unidadeMedida.Situacao != Situacao.ATIVO)
            {
                errors.Add("A unidade de medida informada está inativa e não pode ser utilizada.");
            }
        }

        if (tipoDespesaDTO.IdTipoCodigo > 0)
        {
            var tipoCodigo = await _tipoCodigoRepository.GetById(tipoDespesaDTO.IdTipoCodigo);
            if (tipoCodigo is null)
            {
                errors.Add("O tipo de código informado não existe.");
            }
        }

        if (!string.IsNullOrEmpty(tipoDespesaDTO.Descricao))
        {
            var descricaoNormalizada = NormalizeForComparison(tipoDespesaDTO.Descricao);
            if (await _tipoDespesaRepository.ExistsByDescricaoNormalized(descricaoNormalizada, id))
            {
                errors.Add("Já existe um tipo de despesa cadastrado com esta descrição.");
            }
        }
    }
}

```

```

        if (id.HasValue && existingTipoDespesa is not null
            && existingTipoDespesa.Situacao == Situacao.ATIVO
            && tipoDespesaDTO.Situacao == Situacao.INATIVO
            && await _tipoDespesaRepository.HasDespesasVinculadas(id.Value))
        {
            errors.Add("Não é permitido inativar tipo de despesa com despesas vinculadas.");
        }
    }

    1 referência
    private static string NormalizeForComparison(string descricao)
    {
        return Regex.Replace(descricao.Trim(), "\\s+", " ").ToUpperInvariant();
    }
}

```

```

catch (TipoDespesaValidationException ex)
{
    _response.Code = ResponseEnum.INVALID;
    _response.Data = ex.Errors;
    _response.Message = "Os dados informados para o tipo de despesa são inválidos";
    return BadRequest(_response);
}

```

```

catch (TipoDespesaValidationException ex)
{
    _response.Code = ResponseEnum.INVALID;
    _response.Data = ex.Errors;
    _response.Message = "Os dados informados para o tipo de despesa são inválidos";
    return BadRequest(_response);
}

```



```
if (tipoDespesa.Situacao == Situacao.ATIVO)
{
    var possuiDespesasVinculadas = await _tipoDespesaService.HasDespesasVinculadas(id);
    if (possuiDespesasVinculadas)
    {
        _response.Code = ResponseEnum.INVALID;
        _response.Data = null;
        _response.Message = "Não é possível desativar este tipo de despesa, pois existem despesas vinculadas.";
        return BadRequest(_response);
    }
}
```

```
if (tipoDespesa.Situacao != Situacao.ATIVO)
{
    throw new ArgumentException("TipoDespesa inativo nao pode ser utilizado em novo")
}
```

Assinaturas